CS471 – Web Technologies (Laboratory)	Qassim University College of Computer	Lab 2
		Django and Basic Git operations

This lab session covers the basics of the Django development framework and the Git system. By the end of this session, students should have a very good understanding of how to install and configure a Django application and how to structure it. They should also have practical involvement with the Git system and how to use basic commands to version control their repository.

Pre-lab Preparation:

1. Install Python: Ensure that Python 3.7+ is installed in your system (macOS users shouldn't use their built-in Python version; they should have their own Python environment). Also, ensure to upgrade your pip command by typing '**`python3 -m pip install --upgrade pip`**'
2. Install Git: Ensure that you have git installed. (macOS users may install Xcode or install the commands without Xcode by typing '**`Xcode-select --install`**' in the terminal command line).
3. Since we are using the command line for Git, you must generate SSH keys. Here is an example of how to do that: (https://docs.github.com/en/authentication/cor ssh-key-and-adding-it-to-the-ssh-agent).

```
leen@Mac ~ % xcode-select --install
xcode-select: note: install requested for command line developer tools
leen@Mac ~ % python3 -m pip install --upgrade pip

Requirement already satisfied: pip in /Library/Frameworks/Python.framework/Version
s/3.13/lib/python3.13/site-packages (25.2)
Collecting pip
  Downloading pip-26.0.1-py3-none-any.whl.metadata (4.7 kB)
Downloading pip-26.0.1-py3-none-any.whl (1.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 3.4 MB/s  0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 25.2
    Uninstalling pip-25.2:
      Successfully uninstalled pip-25.2
Successfully installed pip-26.0.1
```

Lab Activities:

Part 1: Django web development framework

Task 1: install Django

Step 1: Once you have Python installed and PIP upgraded on your system, you must create a virtual environment using '**`python3 -m venv djangoenv`**'. This command will create an isolated Python virtual environment for your development. You can do this anywhere, but it ***should not*** be in your local git repository.

Step 2: Once the virtual environment is created, you should activate it. You do this by changing the folder to the virtual environment you just created as follows:

```
> cd path_to_virtual_env/djangoenv
macOS:      > source bin/activate
Windows:    > Scripts\activate
```

Step 3: once activated, we can now install Django

```
> pip install django
```

```
leen@Mac ~ % cd ~
python3 -m venv djangoenv
leen@Mac ~ % cd ~/djangoenv
source bin/activate
(djangoenv) leen@Mac djangoenv % pip install django
Collecting django
  Downloading django-6.0.2-py3-none-any.whl.metadata (3.9 kB)
Collecting asgiref>=3.9.1 (from django)
  Downloading asgiref-3.11.1-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse>=0.5.0 (from django)
  Downloading sqlparse-0.5.5-py3-none-any.whl.metadata (4.7 kB)
Downloading django-6.0.2-py3-none-any.whl (8.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 8.3/8.3 MB 2.6 MB/s  0:00:03
Downloading asgiref-3.11.1-py3-none-any.whl (24 kB)
Downloading sqlparse-0.5.5-py3-none-any.whl (46 kB)
Installing collected packages: sqlparse, asgiref, django
Successfully installed asgiref-3.11.1 django-6.0.2 sqlparse-0.5.5
```

Task 2: create, configure, and organize a new Django project

Step 1: Navigate to the desired location at which you will create the Django web application. For example, 'DjangoProjects' in your home folder.

```
> mkdir ~/DjangoProjects
> cd ~/DjangoProjects/
```

Step 2: Create your Django project as follows:

```
> Django-admin startproject libraryproject
```

```
(djangoenv) leen@Mac djangoenv % mkdir -p ~/DjangoProjects
cd ~/DjangoProjects
(djangoenv) leen@Mac DjangoProjects % django-admin startproject libraryproject
(djangoenv) leen@Mac DjangoProjects % cd ~/DjangoProjects/libraryproject
```

Step 3: run the application to test that everything is working as intended:

```
> python manage.py runserver
```

Step 4: Open your favorite internet browser and navigate to the localhost 'http://127.0.0.1:8000/'. You should see the Django default layout of the Django application.

```
(djangoenv) leen@Mac DjangoProjects % cd ~/DjangoProjects/libraryproject
python3 manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
```

Task 3: configure your Django application

Throughout the lab sessions, the Django application will handle two services (sub-applications). This task guides you in creating the appropriate structure for these two services. Hereafter, we will refer to the **ROOT** folder as the folder that contains the manage.py file.

Step 1: Create some folders in the **ROOT** folder

```
> mkdir apps/ apps/template/ apps/static/
```

```
jangoenv) leen@Mac libraryproject % mkdir -p apps/template apps/static
(djangoenv) leen@Mac libraryproject % cd apps
```

Step 2: create two module services in the apps folder: bookmodule and usermodule

```
> cd apps
> python ../manage.py startapp bookmodule
> python ../manage.py startapp usermodule
```

```
python3 ../manage.py startapp bookmodule
python3 ../manage.py startapp usermodule
```

These commands create two folders inside of the apps folder, and each one has a list of files as follows:

```
├── __init__.py
├── admin.py
├── apps.py
├── migrations
│   └── __init__.py
├── models.py
├── tests.py
└── views.py
```

Step 3: Open the files modules configuration files.

Open the file apps/bookmodule/apps.py and update the module name:

```
name = 'apps.bookmodule'
```

Do the same for the usermodule at apps/usermodule/apps.py

```
name = 'apps.usermodule'
```

Step 4: Update the Django application settings at **ROOT/libraryproject/settings.py**, and add the following to the INSTALLED_APPS array.

```
INSTALLED_APPS = [
    ...,
    'apps.bookmodule',
    'apps.usermodule',
]
```

```
bookmodule/apps.py:    name = 'apps.bookmodule'
usermodule/apps.py:    name = 'apps.usermodule'
(djangoenv) leen@Mac apps % cd ..
```

Step 5: run the server to test that everything works as intended. From the ROOT location of your Django project, do the following:

> python manage.py runserver

Part 2: Git system and connect your online repo with a local repo

Task 1: Create a local git repo.

Step 1: navigate to the **ROOT** folder of your Django application

```
> cd ~/DjangoProjects/libraryproject
```

Step 2: initialize your local Git repository:

```
> git init
(this command will create a git folder)
```

```
[14/Feb/2026 22:17:18] "GET / HTTP/1.1" 200 12068
[14/Feb/2026 22:17:30] "GET / HTTP/1.1" 200 12068
^C%
jangoenv) leen@Mac libraryproject % cd ~/DjangoProjects/libraryproject

(djangoenv) leen@Mac libraryproject % git init
Initialized empty Git repository in /Users/leen/DjangoProjects/libraryproje
```

Step 3: You don't want to add all the files from your local repository to the online one. So, create an ignore file to add all file extensions that the git system will overlook.

```
macOS:      > touch .gitignore & open .gitignore
Windows:    > type NUL > .gitignore & .gitignore
```

Once the file .gitignore is opened in a text editor, add the following lines:

```
*.log
*.pot
*.pyc
```

```
(djangoenv) leen@Mac libraryproject % touch .gitignore
```

```
__pycache__
*.bak
*.py[cod]
*$py.class
pip-log.txt
pip-delete-this-directory.txt
.ipynb_checkpoints
.python-version
celerybeat-schedule.*
# Environments
.env
.venv
env/
venv/
ENV/
env.bak/
venv.bak/
.mypy_cache/
**/migrations
*.DS_Store
**/__pycache__
**/.DS_Store
.history
```

Then, we need to stage the ignore file and let Git know about it.

```
> git add .gitignore
> git commit -m "add ignore file"
```

```
(djangoenv) leen@Mac libraryproject % git add .gitignor
git commit -m "add ignore file"
```

Task 2: connect your local repository with the remote repository you created in Lab 2.

Step 1: before you can do anything, we must first configure git to use the SSH connection with your remote repository. Use the same credentials that you used to create your online repository. Then, you should do the following:

```
> git config --global  user.name "Pixel"
> git config --global  user.email "pixel@webmai
```

```
(djangoenv) leen@Mac libraryproject % git config --global user.name "Leen-abdulazi
z24"
git config --global user.email "leen.abdulaziz.13579@gmail.com"
(djangoenv) leen@Mac libraryproject % git config --global --list | grep user
```

Step 2: Generate the authentication keys. You can visit this **link** for different operating systems, here we use macOS compatibility commands.

```
> ssh-keygen -t ed25519 -C "pixel@webmail.com"
```

```
(djangoenv) leen@Mac libraryproject % ssh-keygen -t ed25519 -C "leen.abdulaziz.135
79@gmail.com" -f ~/.ssh/id_ed25519
```

You will be prompted to enter the key's location. Press the enter/return key on your keyboard to accept the default location, or you must give the full path to your SSH configuration folder.

```
> Enter a file in which to save the key(/Users/Pixel/.ssh/id_ed25519):[Press enter]
```

Then, you will be prompted to enter a passphrase. If you don't want to set it up, you can press the enter/return key anyway.

Step 3: Add the key to ssh-agent

```
> eval "$(ssh-agent -s)"
```

```
(djangoenv) leen@Mac libraryproject % eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

Step 4: Navigate to the online git repository and copy the project link you dedicated to this module. For example, if your repository project is hosted on GitHub, then it should have this pattern:

```
https://github.com/<user_name>/<repo_name>.git
```

Step 4: Add your remote repository to your local git repository:

```
> git remote add origin https://github.com/<user_name>/<repo_name>.git
```

Step 5: Determine which branch your repository project has. Usually, it is called main. Then to the following:

> git fetch origin

```
(djangoenv) leen@Mac libraryproject % git remote -v

origin  https://github.com/Leen-abdulaziz24/labs.git (fetch)
origin  https://github.com/Leen-abdulaziz24/labs.git (push)
```

> git push -u origin main

Step 6: D ed by visiting your remote online repository.

```
(djangoenv) leen@Mac libraryproject % git push -u origin main

To github.com:Leen-abdulaziz24/labs.git
```